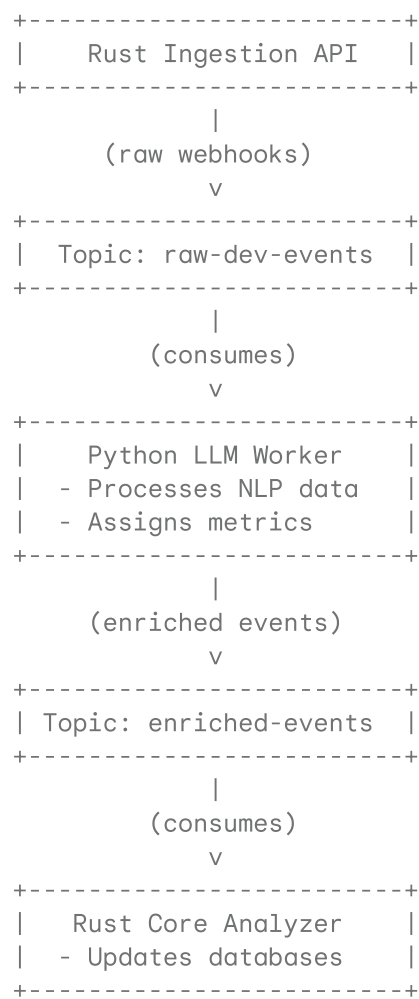


Project Millipede: Streaming ETL & Ingestion Specifications

This document outlines our event-driven Kafka pipeline and the security logic designed for our 1:1 manager notes.

The Chained Consumer Pipeline

We use a **Chained Consumer** design to structure our data processing pipelines. This allows the computationally heavy custom LLM to enrich raw telemetry asynchronously, keeping our databases clean.

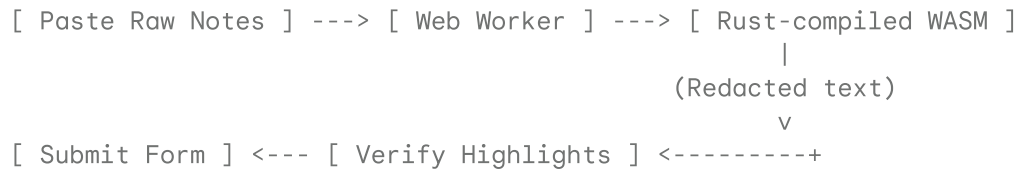


Core Benefits:

- **Single-Write Relational DBs:** PostgreSQL only writes fully processed, clean records. It is never locked up by long-running text analysis updates.
- **Resilient Backpressure:** If a developer triggers a major surge in Git pushes, the Rust Ingestion API publishes these quickly to Kafka. The Python worker continues running inference at its own pace without running out of memory.
- **Log Replayability:** Since Kafka records are immutable, you can upgrade your neural network weights in PyTorch, set your offset to zero, and recalculate your entire historical dashboard.

Ingestion Strategy B: The "Manager-In-The-Loop" 1:1 Portal

Instead of risking data leaks with automated Google Drive scrapers, we use a secure manual portal on the dashboard to ingest 1:1 notes.



Step-by-Step Data Flow:

1. At the end of a 1:1 meeting, you open your secure SolidJS manager dashboard.
2. You paste your raw notes into a local markdown text-area.
3. Your browser offloads the text to a background **Web Worker** running a compiled **Rust WebAssembly** module.
4. The Wasm module strips out real names, replacing them with deterministic pseudonyms (e.g., Alice becomes `Developer_Alpha`, Bob becomes `Developer_Bravo`).
5. The UI displays highlighted redactions, letting you verify the changes before submitting.
6. The sanitized text is sent over your secure mTLS network to Postgres and Kafka, keeping sensitive personal details entirely local to your browser.